

GUÍA DE TALLER Y SUSTENTACIÓN: ARQUITECTURA BACKEND CON POO Y SOLID

Programa: Análisis y Desarrollo de Software (ADSO)

Ficha: 3172526

Instructor: Andrés Moreno Collazos

1. Objetivo del Taller

Demostrar la correcta aplicación del paradigma de Programación Orientada a Objetos (POO) y los principios SOLID en el backend del proyecto formativo de los aprendices. El equipo debe refactorizar y ajustar un módulo central de su software para cumplir con estándares de arquitectura empresarial.

2. Fechas, Horarios y Tiempos de Desarrollo

- **Tiempos de desarrollo:** El taller se debe desarrollar de manera estricta durante las horas de formación asignadas, en los horarios de 6:00 p.m. a 10:00 p.m. y los sábados de 6:30 a.m. a 3:30 p.m.
- **Fecha de entregable y sustentación:** Lunes, 14 de septiembre de 2026.

3. Ajustes Requeridos en el Proyecto (Implementación)

El equipo debe seleccionar un módulo core de su backend e implementar los siguientes conceptos, evidenciables directamente en el código fuente:

A. Paradigma Orientado a Objetos

- **Abstracción:** Definición de interfaces o clases abstractas que establezcan los contratos de los servicios y repositorios de datos.
- **Encapsulamiento:** Protección del estado interno de las entidades. Uso estricto de modificadores de acceso e inyección de dependencias para evitar la exposición de la lógica de negocio.
- **Herencia:** Jerarquías de clases lógicas (relación "es-un") para compartir comportamiento común sin duplicar código.
- **Polimorfismo:** Métodos con un mismo nombre y firma que ejecutan comportamientos distintos dependiendo de la clase hija que los implemente o la estrategia inyectada.

B. Principios SOLID

- **Responsabilidad Única (SRP):** Separación estricta en capas (Controladores, Servicios, Repositorios, Entidades). Ninguna clase debe centralizar múltiples responsabilidades operativas.

- **Abierto/Cerrado (OCP):** El código debe permitir la adición de nuevas funcionalidades (ej. nuevos tipos de usuarios, métodos de pago o reportes) mediante la creación de nuevas clases, sin necesidad de modificar el código existente.
- **Sustitución de Liskov (LSP):** Las clases derivadas deben poder reemplazar a sus superclases sin causar errores en el sistema ni verse forzadas a implementar métodos vacíos o ilógicos.
- **Segregación de Interfaces (ISP):** Creación de interfaces específicas para cada cliente. Se deben evitar las interfaces monolíticas ("clases Dios").
- **Inversión de Dependencias (DIP):** Los módulos de alto nivel (como Controladores) deben depender de abstracciones (Interfaces), no de implementaciones concretas (Repositorios específicos). Se exige el uso de inyección de dependencias en el constructor.

4. Entregables Técnicos Obligatorios

El equipo debe presentar los siguientes artefactos técnicos previo a la exposición:

1. **Diagrama de Arquitectura de Software:** Esquema visual detallando el patrón utilizado en el backend (ej. Patrón en Capas, MVC, Arquitectura Limpia/Hexagonal) y su flujo de comunicación.
2. **Diagrama de Clases (UML):** Modelado exacto del módulo refactorizado, mostrando interfaces, herencia, composición y dependencias.
3. **Código Fuente (Backend):** Acceso al repositorio actualizado en el entorno de desarrollo integrado (IDE) listo para auditoría.

5. Protocolo de Exposición

La sustentación será una auditoría técnica estricta sobre el código fuente y las decisiones arquitectónicas. No se evaluarán presentaciones teóricas.

- **Fase 1: Defensa de la Arquitectura (5 minutos):** Explicación del Diagrama de Arquitectura de Software y el Diagrama de Clases (UML). El equipo debe justificar sus decisiones de diseño y estructuración de carpetas/módulos.
- **Fase 2: Auditoría de Código en Vivo (10 minutos):** Revisión directa en el IDE. El instructor solicitará a los aprendices que ubiquen y expliquen las líneas de código exactas donde aplicaron: Polimorfismo, Separación de responsabilidades (SRP) e Inyección de dependencias (DIP).
- **Fase 3: Resolución de Escenarios (5 minutos):** El instructor planteará un escenario de escalabilidad (ej. "Si el cliente solicita integrar un nuevo proveedor de facturación electrónica, ¿qué clases se deben modificar?"). El equipo debe demostrar que su arquitectura cumple con OCP y absorbe el cambio sin colapsar.

6. Criterios de Evaluación

- **Alineación UML-Código:** El diagrama de clases entregado corresponde con precisión milimétrica a la estructura del backend expuesto en el IDE.

- **Bajo Acoplamiento:** Ausencia total de dependencias directas de instanciación manual (new) en las clases de alto nivel.
- **Defensa Técnica:** Fluidez, uso de terminología arquitectónica adecuada (inyección, contrato, cohesión, abstracción) y dominio total del código por parte de todos los integrantes del equipo.

7. Directrices de Disciplina y Plan de Mejoramiento

Nota de dedicación y enfoque profesional: Es imperativo que el nivel de atención, tiempo y proactividad que frecuentemente se invierte en plataformas de mensajería, juegos y otras actividades de entretenimiento ajenas a los objetivos del programa, sea canalizado con el mismo rigor y disciplina hacia el estudio y la ejecución integral de este taller dentro de las franjas horarias estipuladas.

- **Plan de Mejoramiento:** El grupo que no tenga listo todo lo requerido en el taller para la fecha establecida será objeto de un plan de mejoramiento.
- **Ejecución y sanciones:** Este plan de mejoramiento se ejecutará para su cumplimiento en un término de 7 días. Si no se cumple, se agregarán todas las fallas de inasistencias y el caso pasará directamente a comité para iniciar su debido proceso de retiro.